PSR-Pipeline: An Edge-Oriented Software Stack for sEMG Finger-Intent Decoding in Post-Stroke Neurorehabilitation

PSR Project Team

July 18, 2026

Abstract

Finger-intent decoding from surface electromyography (sEMG) is an appealing control modality for post-stroke neurorehabilitation, but it is difficult to evaluate rigorously when data harmonization, signal conditioning, tensor formatting, model training, and deployment screening live in disconnected scripts. This paper documents the current software portion of PSR-Pipeline, a repository-centered stack that targets five-finger intent decoding from PhysioMio recordings and is intended to feed future embedded rehabilitation hardware based on Raspberry Pi 5-class devices [10, 2, 11]. The repository aligns gesture labels with raw multichannel sEMG, applies a fixed preprocessing chain consisting of a 20–450 Hz zero-phase Butterworth filter, Symlet-4 wavelet denoising, and 200 ms windows with 50% overlap, extracts twelve time- and frequency-domain descriptors per channel-window pair, and standardizes the resulting tensors for LSTM, graph neural network, and 1D CNN model families. The updated benchmark evidence shows that the merged Optuna-tuned CNN-Large student is the strongest committed model artifact, reaching 0.593 exact-match accuracy, 0.798 finger accuracy, 0.714 macro F1, 0.881 macro AUROC, and 0.812 macro AUPRC on the impaired-arm held-out test split, while Nano and Micro students remain within roughly 1.5 percentage points of its exact-match score with much smaller INT8 footprints. The repository also contains healthy-to-impaired transfer-learning summaries, teacher ResNet evaluations, teacher-ensemble distillation code, per-finger threshold tuning, and CPU latency benchmarking utilities, which together strengthen the edge-deployment story. At the same time, the evidence remains software-bound: the repository does not yet commit a distilled-student evaluation bundle produced by the new distillation path, nor does it include an on-device Raspberry Pi 5 timing log or clinical outcome study. We therefore frame PSR-Pipeline as an evidence-constrained software systems paper for arXiv rather than a claim of clinical readiness or definitive state-of-the-art performance.

1 Introduction

Finger-intent prediction from surface electromyography (sEMG) is an attractive interface for post-stroke neurorehabilitation because it offers a non-invasive path from residual muscle activity to assistive control, therapy feedback, and fine-motor retraining [8, 14]. In the broader PSR project, that decoding capability is intended to support compact machine-learning models that can ultimately be served on Raspberry Pi 5-class hardware and connected to a rehabilitation glove. This manuscript focuses only on the software portion of that agenda: the data-processing, modeling, and evaluation stack that turns multichannel sEMG into five-finger intent predictions.

The challenge is not only raw predictive accuracy, but also pipeline reproducibility. Post-stroke sEMG is noisy, patient-specific, and sensitive to recording conditions, so benchmark results depend heavily on how signals are filtered, denoised, segmented, labeled, tensorized, and split across patients.

Reviews of sEMG-based gesture recognition consistently note that preprocessing design, feature representation, and real-time deployment constraints can matter as much as the final classifier family [7, 9]. A repository that exposes only isolated models without a stable preprocessing and evaluation path is therefore difficult to audit and even harder to extend toward embedded rehabilitation use.

PSR-Pipeline addresses that gap by collecting the end-to-end software workflow in one codebase [10]. The current repository includes PhysioMio ingestion, gesture-to-finger label mapping, a fixed signal-conditioning stack, window-level feature extraction, tensor adapters, LSTM and GNN baselines, a merged 1D CNN student/teacher suite, Optuna-based hyperparameter optimization, healthy-to-impaired transfer-learning summaries, and benchmark artifacts stored as committed metric files. Recent repository updates also added teacher-ensemble distillation, per-finger threshold tuning, and latency benchmarking utilities, which materially improve the project's edge-oriented software story even when the outputs of those utilities are not yet fully committed as benchmark bundles.

The central story of this paper is therefore systems-oriented rather than novelty-driven. We do not present a new decoding architecture, nor do we claim that the repository already demonstrates end-to-end rehabilitation hardware validation. Instead, we document a reusable software stack whose main value lies in making engineering assumptions explicit: which recordings are used, how they are preprocessed, how tensors are shared across model families, which metrics are computed, and which deployment-facing hooks are already present in code.

This updated paper makes four contributions. First, it formalizes the current repository pipeline from PhysioMio ingestion through preprocessing, feature extraction, tensor adaptation, model fitting, and deployment-screening utilities. Second, it consolidates the merged benchmark evidence into one software-facing narrative, including the optimized CNN student sweep, teacher evaluations, legacy direct baselines, and transfer-learning summaries. Third, it identifies what the updated repository now supports for edge-oriented experimentation, including compact student architectures, teacher-ensemble distillation, threshold calibration, and latency measurement. Fourth, it draws a careful boundary around what is still missing, especially committed distilled-student outputs, on-device Raspberry Pi 5 validation, broader external baselines, and clinical evidence.

2 Related Work

sEMG has long been studied as a non-invasive control channel for rehabilitation, prosthetics, and human-machine interaction, and stroke-specific reviews show that sEMG-driven interventions can support meaningful upper-limb rehabilitation when signal interpretation is reliable enough for assistive use [8, 14]. That literature also makes clear that sensing and decoding should not be separated conceptually: the clinical value of an sEMG system depends on acquisition quality, label definition, processing latency, and how well predictions align with therapy needs.

Recent reviews of gesture recognition from sEMG emphasize two recurring design tensions [7, 9]. The first is representation choice: end-to-end deep models are increasingly popular, but carefully chosen handcrafted descriptors remain attractive when data are limited, interpretability matters, or deployment hardware is constrained. The second is systems realism: many published results focus on offline accuracy while leaving preprocessing assumptions, thresholding strategy, or inference-time constraints underspecified. A repository-centered paper can therefore contribute value even without proposing a new classifier, provided it makes those choices explicit and reproducible.

Sequence-oriented deep models remain natural baselines for windowed biosignal decoding because

they can aggregate temporal evidence across successive frames. LSTM-based models explicitly model temporal dependencies in noisy sequential measurements [5], while compact 1D CNNs emphasize local temporal motifs and can be aggressively compressed for efficient inference. Graph neural networks offer a complementary viewpoint by representing windowed observations as nodes connected through a learned message-passing structure rather than as a purely linear sequence [6, 3, 12]. The current repository follows all three lines so that architectural trade-offs can be studied under a common preprocessing and evaluation regime.

Deployment-oriented optimization provides a final piece of context for this paper. Knowledge distillation is a standard way to transfer behavior from larger teachers into smaller student models [4], and Optuna is widely used for practical black-box hyperparameter search [1]. What distinguishes PSR-Pipeline from a typical single-model benchmark is therefore not a new recurrent, convolutional, or graph formulation, but a software stack that places feature extraction, architecture comparison, transfer learning, distillation hooks, and deployment proxies inside one auditable repository.

3 Method

3.1 Overview

PSR-Pipeline implements a modular decoding workflow that starts from raw multichannel sEMG and gesture labels and ends with either five finger logits or downstream evaluation artifacts. The software stack is easiest to read as four connected bands: data harmonization and ingestion, signal processing and feature extraction, model fitting, and evaluation or deployment-facing analysis. Figure 1 summarizes that workflow at the paper level. The figure includes modeling and deployment-oriented blocks that are present in the repository codebase, but the empirical claims in this paper remain restricted to committed software artifacts.

3.2 Data harmonization and ingestion

The repository loader builds processed datasets directly from raw PhysioMio parquet files when cached tensors are missing. It searches the raw data tree, groups files by patient, aligns contiguous stretches of the same `movement_type`, maps each label to a five-bit finger target, discards segments shorter than 200 samples, and then applies the full preprocessing pipeline to each usable segment. These ingest operations define the effective task boundary: which arm is selected, how gestures become multilabel targets, and whether repeated data from the same patient can leak across train and test.

The default configuration uses PhysioMio recordings sampled at 2000 Hz, selects the impaired arm unless the user explicitly changes the arm split, and performs a deterministic patient-level 70/10/20 train/validation/test split. Processed samples are padded to a dataset-wide maximum window count so they can be stacked into tensors. The resulting dataset is therefore not just a collection of windows; it is a patient-split, segment-level benchmark derived from raw multichannel recordings.

3.3 Signal preprocessing

The preprocessing module is explicitly parameterized by a frozen configuration object in the repository. Raw sEMG first passes through a fourth-order Butterworth band-pass filter with 20–450

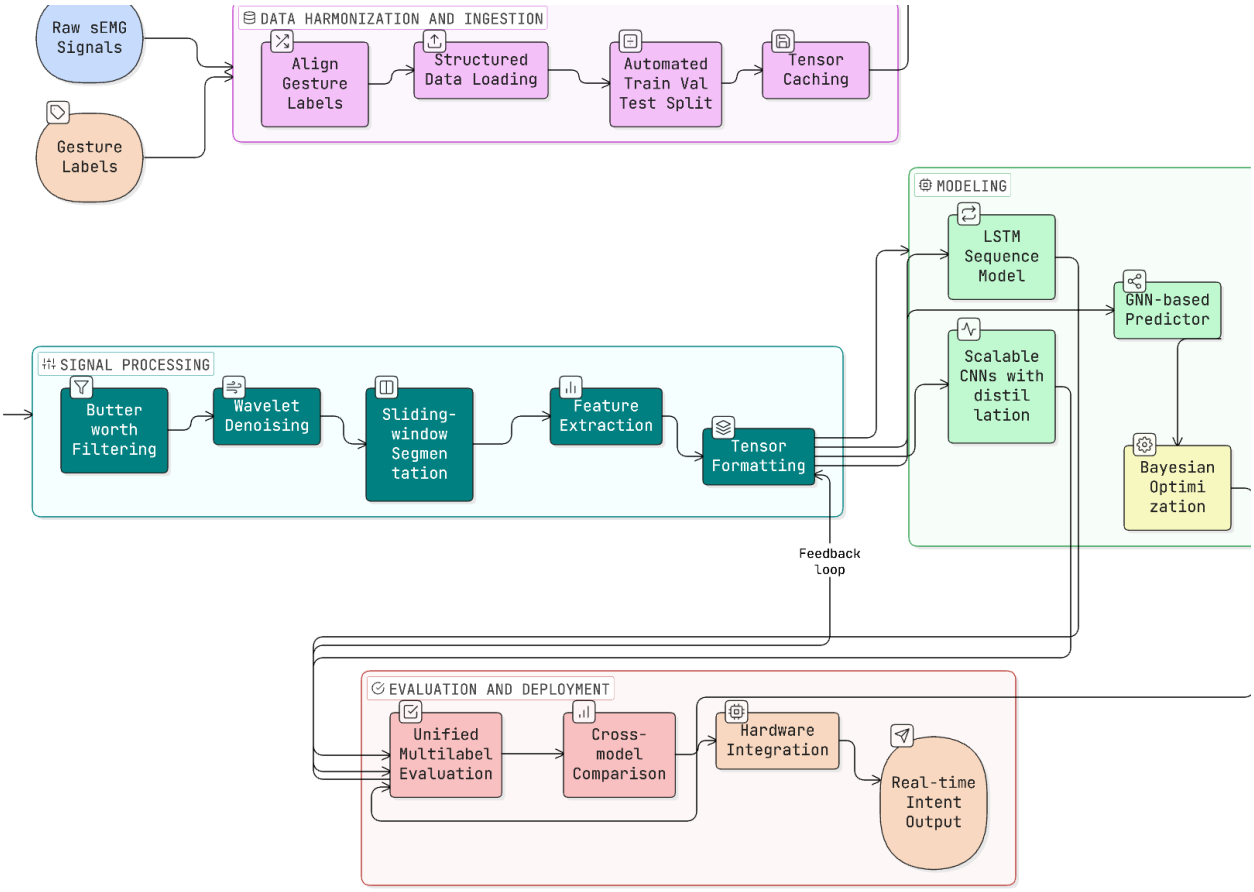

Figure 1: Paper-level view of the current software stack. The committed repository covers data harmonization, preprocessing, feature extraction, tensor formatting, multiple predictor families, unified evaluation, and deployment-facing utilities such as threshold tuning and latency benchmarking. Hardware integration and real-time output are project targets rather than experimentally validated claims in this manuscript.

Hz cutoffs and zero-phase forward-backward application. This stage suppresses low-frequency motion artifacts and high-frequency electrical noise while preserving time alignment across channels.

The filtered signal is then denoised with wavelet soft-thresholding. The default configuration uses a Symlet-4 basis, four decomposition levels, universal threshold selection, and a median-based noise estimate. Although the repository does not yet include an ablation isolating this choice, the denoising rule is explicit, versioned, and reusable across all model families.

After denoising, the continuous recording is segmented into 200 ms windows with 50% overlap and no zero-padding. These settings balance temporal responsiveness against the sample count required for stable summary statistics. They are also consistent with the repository's edge-oriented framing: the software stack targets compact downstream inference and therefore uses short windows that could plausibly support reactive control.

3.4 Window-level feature representation

Each channel-window pair is summarized by twelve handcrafted descriptors spanning time and frequency domains. The time-domain set includes root mean square, mean absolute value, integrated EMG, waveform length, variance, zero crossings, slope sign changes, and Willison amplitude. The frequency-domain set includes mean frequency, median frequency, spectral entropy, and total power, with spectral quantities estimated from Welch periodograms using `nperseg = min(256, len(x))` [13]. Degenerate spectra are zeroed explicitly to avoid unstable feature values. The resulting tensor has shape (C, W, F), where C is the number of channels, W is the number of windows, and $F = 12$ is the feature count.

3.5 Tensor formatting and shared adapter

The repository standardizes cross-model comparison with one shared adapter. It transforms (C, W, F) or (N, C, W, F) tensors into $(N, W, C \times F)$ sequences, where windows act as time steps and all channel features are concatenated into one vector per step. This interface decision is small but important because it lets recurrent, convolutional, and graph-based predictors share the same derived representation without duplicating preprocessing code.

For the CNN branch, the adapted tensor is subsequently permuted to a channels-first layout. The merged CNN stack treats the 64×12 channel-feature grid as 768 input channels for temporal `Conv1d` layers, which preserves a common upstream pipeline while letting the convolutional branch operate as a lightweight sequence model over window order.

3.6 Predictor families

The LSTM branch applies two stacked recurrent layers with hidden sizes 128 and 256, followed by a 128-unit fully connected layer and dropout before five output logits. This branch emphasizes temporal accumulation across windows and serves as a strong sequence baseline for the direct benchmark.

The GNN branch interprets each time window as a node in a graph. The implementation supports GCN, GraphSAGE, and GAT operators [6, 3, 12], and the benchmarked path constructs a dense graph over windows within each sample before graph-level pooling and readout. This branch injects an explicit structural prior: windows are related observations rather than only a flat sequence.

The CNN branch is the most elaborate family in the updated repository. It includes five student variants ranging from Nano to XLarge, each built around a cheap 1×1 projection, temporal convolution blocks, adaptive pooling, and a small fully connected head. The accompanying README reports approximate footprints from 54K parameters and 53 KB INT8 for Nano up to 1.62M parameters and 1.58 MB INT8 for XLarge, while the teacher family includes 1D ResNet-50, ResNet-101, and ResNet-152 variants. The key software idea is that all of these models sit behind the same input adaptation pathway, so increasing model capacity does not require changing ingestion or preprocessing logic.

3.7 Training, tuning, and evaluation protocol

The training utilities unify optimization and reporting across model families. The repository uses binary cross-entropy with logits for five-label prediction, Adam-based optimization, plateau-triggered

learning-rate reduction, early stopping, checkpointing, and saved metric curves. Evaluation reports exact-match accuracy, finger-level averaged accuracy, macro precision, macro recall, macro F1, macro AUROC, and macro AUPRC. The metrics module also stores per-finger scores and task-level curve data, which enables both aggregate and finger-wise analysis after training.

Hyperparameter optimization is a first-class component rather than an afterthought. The merged CNN stack uses Optuna to search architecture and training choices for each student size, and its score combines validation accuracy with a latency-based penalty relative to a baseline timing reference [1]. Trial-level early stopping is used during search so poorly performing candidates terminate quickly. The repository also contains two deployment-oriented utilities: a teacher-ensemble distillation path that can fuse multiple teachers either equally or by per-finger validation performance, and a threshold-tuning script that sweeps per-finger thresholds over a validation grid to maximize F1. A separate latency benchmark reports mean, median, and 95th-percentile inference time on a chosen device. Together, these components extend the repository beyond a pure offline benchmark and toward a practical software stack for edge screening.

3.8 Transfer-learning path

In addition to direct impaired-arm benchmarks, the repository stores two-stage summaries for CNN and LSTM models that pretrain on healthy-arm data and finetune on impaired-arm data. These artifacts matter for post-stroke applications because impaired-arm data are often scarcer and harder to collect than healthy data. Although the repository does not yet harmonize every transfer run into the same directory structure as the direct benchmark, the committed summaries are sufficient to study whether the two-stage path helps each model family.

4 Experiments

4.1 Dataset and split construction

All benchmark artifacts analyzed in this paper come from the repository's PhysioMio processing path [2, 10]. The loader searches raw parquet files, extracts contiguous movement segments, maps gesture labels to five-bit finger targets, preprocesses each segment, and stores processed tensors as PyTorch split files.

The split logic is patient-level rather than segment-level. Patients are shuffled with a fixed seed and divided into 70% train, 10% validation, and 20% test partitions, which reduces identity leakage between train and test segments. The default loader configuration targets impaired-arm recordings unless the user explicitly changes the arm-split setting, so the main benchmark should be interpreted as an impaired-arm post-stroke decoding study. The two-stage transfer summaries use the same software pathway with healthy-arm pretraining followed by impaired-arm finetuning.

4.2 Evidence scope

The repository exposes a richer experimental surface than a single comparison table. In addition to direct LSTM, CNN, and GNN runs, the codebase contains optimized CNN student and teacher evaluations, healthy-to-impaired pretraining and finetuning summaries, optional threshold tuning, teacher-ensemble distillation, and latency benchmarking. However, not all branches are represented by equally complete committed result bundles. This paper therefore separates three evidence tiers:

Table 1: Held-out test performance from the repository metric files. The first three rows are historical direct baselines; the final row is the strongest merged optimized CNN artifact. Bold numbers mark the best score in each column.

Model	Exact-match	Finger Acc.	Macro F1	Macro AUROC	Macro AUPRC
LSTM	0.545	0.784	0.705	0.858	0.754
GNN	0.448	0.683	0.706	0.787	0.776
CNN legacy	0.442	0.694	0.676	0.767	0.764
CNN-Large	**0.593**	**0.798**	**0.714**	**0.881**	**0.812**

direct committed test metrics, committed auxiliary summaries such as transfer learning and CNN evaluation sweeps, and code-only deployment utilities whose behavior is documented but whose outputs are not yet comprehensively committed.

The repository has also evolved over time. Historical direct baselines remain under `metrics/`, while newer CNN outputs live under `models/CNN/evaluations/`. We keep both because they jointly show how the software stack matured from an initial cross-family benchmark into a stronger edge-oriented CNN branch. When a newer optimized artifact supersedes an older baseline, the paper says so explicitly rather than averaging across incompatible runs.

4.3 Evaluation protocol

The repository defines the task as five-label multilabel classification with a nominal decision threshold of 0.5 on the held-out test split. For transparency, the metric implementation records both exact-match accuracy and finger-wise summaries, which is important because different models can trade precision for recall while achieving similar per-label correctness. The headline tables in this paper use the metrics as stored in the committed result files. Threshold-tuning utilities are described in the methods and discussion but are not used to create new manuscript numbers.

This paper follows an evidence-constrained writing policy. Every headline claim is tied to a committed code path, metric file, or repository summary artifact. Because the repository does not yet store a full ablation package, an on-device Raspberry Pi 5 benchmark log, or a polished distilled-student result bundle, we deliberately avoid module-level causality claims and instead focus on benchmark-level observations that are directly supported by saved outputs.

5 Results

The updated repository no longer reads as a simple LSTM-versus-GNN benchmark. Table 1 shows four distinct reference points: the historical LSTM, GNN, and CNN direct baselines stored under `metrics/`, and the strongest merged optimized CNN artifact, Optuna CNN-Large, stored under `models/CNN/evaluations/`. The optimized CNN-Large model is the strongest committed result bundle overall, exceeding all three historical baselines on exact-match accuracy, finger accuracy except for a negligible margin against CNN-Base, macro F1, macro AUROC, and macro AUPRC.

The change from the legacy CNN baseline to the optimized CNN-Large artifact is particularly informative. Exact-match accuracy rises from 0.442 to 0.593, macro F1 from 0.676 to 0.714, macro AUROC from 0.767 to 0.881, and macro AUPRC from 0.764 to 0.812. This is not a minor bookkeeping difference. It shows that once the repository's modular CNN stack, tuned architecture search, and

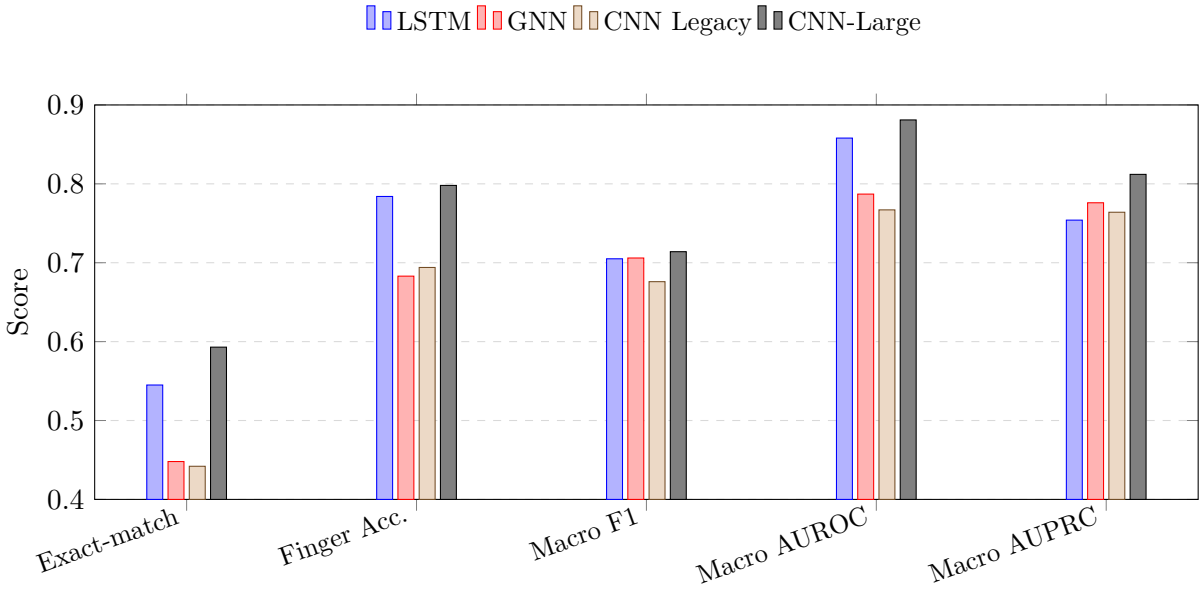

Figure 2: Aggregate-metric comparison across historical direct baselines and the strongest optimized CNN artifact. The merged CNN-Large model shifts the frontier relative to the older direct benchmark files.

Table 2: Merged CNN student sweep. Parameter counts and INT8 sizes come from the repository CNN README; metrics come from committed evaluation bundles.

Variant	Params	INT8 Size	Exact-match	Finger Acc.	Macro F1	Macro AUPRC
Nano	54K	53 KB	0.580	0.794	0.698	0.758
Micro	158K	154 KB	0.578	0.794	0.697	0.793
Base	390K	381 KB	0.575	**0.798**	0.707	0.775
Large	803K	784 KB	**0.593**	0.798	**0.714**	**0.812**
XLarge	1.62M	1.58 MB	0.551	0.794	0.690	0.726

updated evaluation bundle are taken into account, the software story becomes substantially more favorable to compact convolutional models than the older README-era comparison suggested.

The optimized CNN student sweep in Table 2 clarifies the deployment-oriented trade-off. CNN-Large is best on exact-match accuracy, macro F1, macro AUROC, and macro AUPRC, while CNN-Base is effectively tied on finger accuracy. At the same time, the Nano and Micro students remain close to the larger models despite much smaller footprints. This is the key edge-computing result in the repository: the compact students do not collapse in quality when the model is pushed into Raspberry Pi-friendly size ranges.

Teacher results provide a useful reference for how far the compact students can go without explicit deployment measurements. Among the committed teacher checkpoints, ResNet-152 is strongest with 0.588 exact-match accuracy, 0.698 macro F1, 0.870 macro AUROC, and 0.787 macro AUPRC. The optimized CNN-Large student slightly exceeds that teacher on exact-match accuracy, macro F1, macro AUROC, and macro AUPRC while remaining far smaller than a deep ResNet-style model. This does not prove that distillation is already delivering the gains, because the repository does not yet commit a comparable distilled-student result set; it does show that the current tuned student

Table 3: Committed healthy-to-impaired transfer-learning summaries. Values are taken directly from the repository summary files.

Model / Stage	Exact-match	Finger Acc.	Macro F1	Macro AUROC	Macro AUPRC
CNN pre.	0.465	0.760	0.664	0.833	0.699
CNN fine.	**0.585**	**0.800**	**0.680**	**0.863**	**0.767**
LSTM pre.	0.523	0.752	**0.679**	**0.858**	**0.773**
LSTM fine.	**0.574**	**0.777**	0.660	0.830	0.736

Table 4: Per-finger F1 score by model family. The optimized CNN-Large replaces the legacy direct CNN row for the finger-level comparison. Bold numbers mark the strongest model for each finger.

Finger	LSTM	GNN	CNN-Large
Thumb	0.716	**0.776**	0.729
Index	0.723	0.696	**0.750**
Middle	**0.712**	0.689	0.698
Ring	**0.696**	0.688	0.695
Little	0.680	0.678	**0.696**

family is already competitive against its teacher line.

Transfer-learning summaries in Table 3 show that healthy-to-impaired staging helps the CNN family more clearly than the LSTM family. CNN exact-match accuracy improves from 0.465 in healthy pretraining evaluation to 0.585 after impaired-arm finetuning, and finger accuracy rises from 0.760 to 0.800. LSTM exact-match accuracy also improves after finetuning, but its macro AUROC and macro AUPRC decline relative to the healthy pretraining evaluation summary. The transfer takeaway is therefore asymmetric: the staged path looks clearly beneficial for the CNN branch, while the LSTM branch appears more sensitive to domain shift or optimization regime.

Finger-wise scores show that the ranking is not uniform across all outputs. Table 4 compares the LSTM direct baseline, the GNN direct baseline, and the best optimized CNN student. The GNN remains strongest on the thumb label, the optimized CNN-Large leads on index and little finger, and the LSTM remains slightly strongest on middle and ring finger. This pattern suggests that aggregate CNN gains do not erase all output-specific variation; different inductive biases still matter at the finger level.

The results should nevertheless be read with an evidence boundary in mind. The repository now includes code for teacher-ensemble distillation, per-finger threshold tuning, and CPU latency measurement, but it does not yet commit a mature result bundle produced by those new utilities, nor does it store on-device Raspberry Pi 5 timings. The current paper can therefore support a stronger software-stack narrative than the previous draft, but not a full claim of embedded deployment readiness.

6 Discussion

The strongest conclusion supported by the updated evidence is that PSR-Pipeline has matured into a credible edge-oriented software stack for sEMG finger-intent decoding, not just a loose collection of model experiments. Another researcher or engineer can now trace the workflow from raw PhysioMio recordings through preprocessing, feature extraction, tensor adaptation, direct baselines, optimized

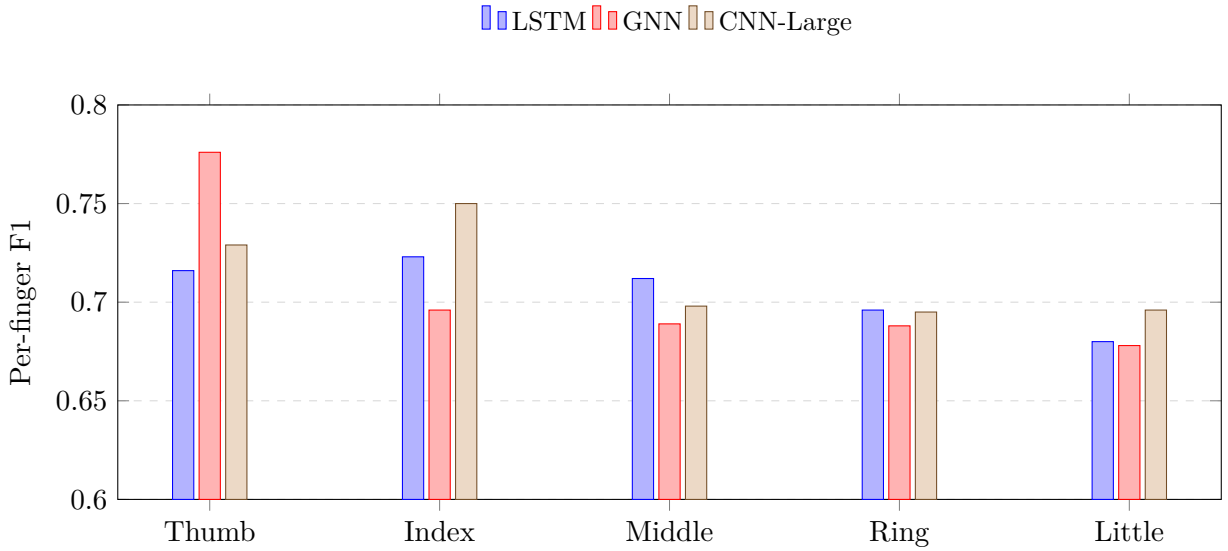

Figure 3: Per-finger F1 comparison derived from the committed metric files. Aggregate CNN gains coexist with output-specific strengths for the LSTM and GNN baselines.

CNN students, teacher references, transfer-learning summaries, and deployment-facing utilities without reconstructing the pipeline by hand.

The updated benchmark also changes the model-level interpretation. In the earlier draft, the repository looked like a trade-off between an LSTM with balanced discrimination and a GNN with stronger recall-sensitive summaries. That story is now incomplete. Once the merged CNN student sweep is included, the strongest committed artifacts are compact convolutional models that outperform the older direct baselines on every headline metric while remaining small enough to fit the intended embedded setting after quantization. This is exactly the kind of repository evolution that a software paper should capture.

The transfer-learning summaries point toward a second useful lesson. Healthy-to-impaired staging appears substantially more promising for the CNN branch than for the LSTM branch. That asymmetry may reflect architecture-specific robustness, optimization differences, or the way the shared feature representation interacts with each model family. The repository does not yet include the ablations needed to explain the mechanism, but it does provide enough evidence to motivate transfer learning as a real software direction rather than a speculative aside.

The repository's deployment story is also more coherent than before, but it is still incomplete. The codebase now includes teacher-ensemble distillation, threshold calibration, and latency benchmarking, and the student model sizes are explicitly framed around Raspberry Pi 5-class constraints. Those are meaningful steps toward embedded inference. However, the paper still cannot claim a validated deployment path because the committed artifact set does not yet include end-to-end Raspberry Pi measurements, power or thermal observations, or a reproducible distilled-student benchmark pack aligned with the rest of the evaluation tree.

Several limitations remain and should be read plainly. First, the benchmark still centers on a single dataset and one main preprocessing configuration, with no committed ablation package isolating the contribution of filtering, denoising, feature design, threshold choice, or graph connectivity. Second, the GNN implementation uses dense window graphs, which may not scale gracefully as sequence length grows. Third, the transfer-learning summaries are committed as high-level JSON rather than

full rerunnable experiment bundles with harmonized plotting and checkpoint metadata. Fourth, the broader rehabilitation objective still lacks the evidence that matters most for translation: on-device timing, robust behavior across users and sessions, and human-subject evaluation of whether predicted intent meaningfully improves therapy.

Taken together, these limitations do not weaken the value of the current paper as a software manuscript for arXiv. They define the next layer of work. The clearest path forward is to push the same artifact discipline into distilled-student evaluation, Raspberry Pi 5 benchmarking, and broader cross-patient or cross-session validation so that the software stack can be connected to a fully supported deployment paper later.

7 Conclusion

This paper documents PSR-Pipeline as an edge-oriented software stack for sEMG-based finger-intent prediction in post-stroke neurorehabilitation. The contribution is not a new standalone network, but a reproducible repository structure that makes ingestion, preprocessing, feature extraction, tensor adaptation, architecture comparison, transfer learning, and deployment-screening utilities explicit and reusable.

The strongest current evidence comes from the merged optimized CNN student family, especially CNN-Large, which improves substantially over the older direct baselines while preserving a compact deployment-oriented footprint. The repository also now supports teacher models, healthy-to-impaired transfer summaries, distillation hooks, threshold tuning, and latency benchmarking, which together make the software story more complete than in the earlier draft. The main remaining gap is not another round of prose polishing; it is evidence. The next step is to commit the same level of artifact completeness for distilled students and Raspberry Pi 5 validation that the repository already provides for its core benchmark path.

References

[1] Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2019.

[2] Formove AI. PhysioMio. https://huggingface.co/datasets/formove-ai/physiomio, 2026. Dataset page, accessed July 18, 2026.

[3] William L. Hamilton, Rex Ying, and Jure Leskovec. Inductive representation learning on large graphs. In *Advances in Neural Information Processing Systems*, 2017.

[4] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2015.

[5] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9 (8):1735–1780, 1997.

[6] Thomas N. Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations*, 2017.

[7] Wei Li, Ping Shi, and Hongliu Yu. Gesture recognition using surface electromyography and deep learning for prostheses hand: State-of-the-art, challenges, and future. *Frontiers in Neuroscience*, 15:621885, 2021. doi: 10.3389/fnins.2021.621885.

[8] Maria Munoz-Novoa, Morten B. Kristoffersen, Katharina S. Sunnerhagen, Autumn Naber, Margit Alt Murphy, and Max Ortiz-Catalan. Upper limb stroke rehabilitation using surface electromyography: A systematic review and meta-analysis. *Frontiers in Human Neuroscience*, 16:897870, 2022. doi: 10.3389/fnhum.2022.897870.

[9] Sike Ni, Mohammed A. A. Al-qaness, Ammar Hawbani, Dalal Al-Alimi, Mohamed E. Abd Elaziz, and Ahmed A. Ewees. A survey on hand gesture recognition based on surface electromyography: Fundamentals, methods, applications, challenges and future trends. *Applied Soft Computing*, 166:112235, 2024. doi: 10.1016/j.asoc.2024.112235.

[10] Post-Stroke Rehabilitation Project. PSR-Pipeline. https://github.com/post-stroke-rehab/psr-pipeline, 2026. GitHub repository, accessed July 18, 2026.

[11] Raspberry Pi Ltd. Introducing: Raspberry Pi 5! https://www.raspberrypi.com/news/introducing-raspberry-pi-5/, 2023. Official product announcement, accessed July 18, 2026.

[12] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. Graph attention networks. In *International Conference on Learning Representations*, 2018.

[13] Peter D. Welch. The use of fast fourier transform for the estimation of power spectra: A method based on time averaging over short, modified periodograms. *IEEE Transactions on Audio and Electroacoustics*, 15(2):70–73, 1967.

[14] Mingde Zheng, Michael S. Crouch, and Michael S. Eggleston. Surface electromyography as a natural human-machine interface: A review. *arXiv preprint arXiv:2101.04658*, 2021.